

Correction – TD/TP 3

Classes, invariants et RAI

MAM4 – Programmation C++

Cette correction donne directement les réponses et le code attendu. Les traces de construction et de destruction servent seulement aux observations : elles peuvent être retirées de la version finale.

1. Quand IntArray est-il correct ?

Les règles retenues dans ce TD sont :

$$0 \leq \text{size} \leq \text{capacity}.$$

Si `capacity == 0`, alors `data == nullptr`. Si `capacity > 0`, `data` désigne une zone de `capacity` entiers. Les éléments utilisés sont `data[0]` à `data[size - 1]` lorsque `size > 0`.

L'état interne vide `size = 0`, `capacity = 0`, `data = nullptr` est valide. Le constructeur public peut néanmoins refuser `IntArray{0}` : c'est un choix d'interface pour la capacité initiale demandée.

Réponse. Un **invariant** est une règle qui doit toujours rester vraie pour que l'objet soit correct.

Avec la structure publique initiale, ces trois corruptions compilent :

```
values.size = values.capacity + 1;
values.capacity = 2; values.size = 8;
values.capacity = 8; values.data = nullptr;
```

Dans chaque cas, au moins une règle précédente devient fausse.

2. Les données deviennent privées

Voici la déclaration finale de la classe dans `int_array.hpp` :

```
#ifndef SEANCE3_INT_ARRAY_HPP
#define SEANCE3_INT_ARRAY_HPP

#include <cstddef>

class IntArray {
public:
    explicit IntArray(std::size_t capacity);
    ~IntArray();

    IntArray(const IntArray&) = delete;
    IntArray& operator=(const IntArray&) = delete;

    void push_back(int value);
    int at(std::size_t index) const;
    int sum() const;
    void fill(int value);
    std::size_t size() const;
    std::size_t capacity() const;

private:
    std::size_t capacity_{};
    std::size_t size_{};
    int* data_{nullptr};
};
```

```
};

#endif
```

Réponse. `private` empêche le code extérieur de modifier directement `capacity_`, `size_` et `data_`. Il doit passer par les fonctions publiques.

Un `IntArray` doit être utilisable dès sa création. Avec une fonction `initialize`, un objet pourrait exister sans mémoire entre sa déclaration et l'appel d'initialisation, ou cet appel pourrait être oublié.

À retenir. Le constructeur remplace la séquence « créer puis initialiser » par une seule opération qui produit directement un objet correct.

3. Constructeurs

Dans `int_array.cpp`, la capacité est vérifiée avant l'allocation :

```
#include "int_array.hpp"

#include <stdexcept>

std::size_t checked_capacity(std::size_t capacity)
{
    if (capacity == 0) {
        throw std::invalid_argument{
            "capacity must be positive"};
    }
    return capacity;
}

IntArray::IntArray(std::size_t capacity)
    : capacity_{checked_capacity(capacity)},
      size_{0},
      data_{new int[capacity_]}{}
{}
```

Réponse. `checked_capacity` est appelée avant `new int[...]`. Si la capacité vaut zéro, l'exception est lancée avant de réserver la mémoire.

```
IntArray values{8};           // capacité 8
IntArray impossible{0};       // std::invalid_argument
```

Réponse. `explicit` empêche une conversion automatique comme transformer silencieusement l'entier 20 en `IntArray{20}`.

Réponse. La capacité nulle n'est pas interdite dans tous les états internes. Seule cette construction publique la refuse; l'état vide `0 / 0 / nullptr` reste valide.

4. Lire ou modifier

Les méthodes d'observation sont constantes :

```
std::size_t IntArray::size() const
{
    return size_;
}

std::size_t IntArray::capacity() const
{
    return capacity_;
}

int IntArray::at(std::size_t index) const
{
    if (index >= size_) {
        throw std::out_of_range{"IntArray::at"};
    }
    return data_[index];
}

int IntArray::sum() const
{
    int result{};
    for (std::size_t i{}; i < size_; ++i) {
        result += data_[i];
    }
    return result;
}

void IntArray::fill(int value)
{
    for (std::size_t i{}; i < size_; ++i) {
        data_[i] = value;
    }
}
```

Réponse. `const` après une méthode signifie que cette méthode ne modifie pas l'objet sur lequel elle est appelée. Ainsi `size()` fonctionne sur un `const IntArray`, tandis que `push_back()` est refusée.

Les écritures

```
int sum(const IntArray& values); // fonction libre
int IntArray::sum() const;      // méthode
```

expriment la même idée : `sum` lit l'objet sans le modifier.

La méthode d'ajout reprend la logique de la séance 2 avec les nouveaux noms :

```
void IntArray::push_back(int value)
{
    if (size_ == capacity_) {
        const std::size_t new_capacity{2 * capacity_};
        int* new_data{new int[new_capacity]{};};

        for (std::size_t i{}; i < size_; ++i) {
            new_data[i] = data_[i];
        }

        delete[] data_;
        data_ = new_data;
        capacity_ = new_capacity;
    }

    data_[size_] = value;
    ++size_;
}
```

5. Destructeur et durée de vie

```
IntArray::~IntArray()
{
    delete[] data_;
}
```

Avec des traces temporaires, l'ordre attendu est :

```
construction 4
construction 8
destruction 8
destruction 4
```

Réponse. Les objets locaux sont détruits dans l'ordre inverse de leur construction. Le destructeur est également appelé lorsqu'un objet déjà construit quitte sa portée à cause d'un `return` ou d'une exception.

Personne ne doit appeler `destroy(values)` : cette fonction a disparu. Le destructeur, appelé automatiquement, exécute `delete[]`.

À retenir. La mémoire est liée à la durée de vie de `IntArray` : le constructeur la réserve et le destructeur la libère. C'est ici l'idée du RAIL.

6. Pourquoi interdire provisoirement la copie ?

Avant les deux lignes `= delete`, l'instruction

```
IntArray copy{values};
```

recopie notamment la valeur de `data_`. Par conséquent :

```
values.data_ ----+
                  +----> [ 42 | ... ]
copy.data_  -----+
```

Réponse. Les deux objets désignent la même allocation. Leurs deux destructeurs essaient ensuite de la libérer : AddressSanitizer signale une double libération.

`size_` et `capacity_` contiennent directement des nombres. `data_` contient seulement une adresse : copier cette adresse ne crée pas un second tableau.

```
IntArray(const IntArray&) = delete;
IntArray& operator=(const IntArray&) = delete;
```

Ces déclarations interdisent provisoirement la construction par copie et l'affectation par copie.

À retenir. Copier un pointeur copie l'adresse, pas les données pointées.

Bilan

Écriture / mécanisme	Effet recherché
<code>private</code>	empêche les modifications directes des données internes
<code>constructeur</code>	crée un objet correct et réserve sa mémoire
<code>destructeur</code>	libère automatiquement la mémoire
<code>const</code> après une méthode	indique une méthode de lecture
<code>= delete</code>	interdit provisoirement une opération
copie d'un pointeur	copie l'adresse, pas les données

Conclusion. `IntArray` protège maintenant ses données internes, réserve sa mémoire lors de sa construction et la libère lors de sa destruction. La copie automatique étant incorrecte, elle reste interdite jusqu'à la séance suivante.